

## Design Document

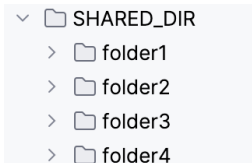
The objective of this assignment is to enhance the existing file transfer mechanism to incorporate parallelism by splitting files into fixed-size chunks. Instead of downloading a file from a single node, peer nodes will concurrently download these chunks from multiple peers, reconstructing the original file. The integrity of each chunk will be verified to ensure the downloaded file matches the original.

### Project Structure

- **SHARED\_DIR**
  - A directory of directories that can be used by the clients to register on the Indexing Server. User can give their own directory for registry. This can be created using `generate_shared_dir.sh` script.
- **docs**
  - documents, manuals, reports.
- **lib**
  - contains the jar file being used in the maven project. This is used while executing the program.
- **src – source code**
  - **com.aos**
    - **PeerInterface.java** – defines the main download logic for requesting and downloading files on the peer side in a serial or parallel fashion(as specified by the user)
    - **IndexServer.java** – holds the logic for Registering the Indexing server on port 3788, binding it and starting the server.
    - **IndexServerInterface.java** – Interface extending Remote, with applicable methods for registering or searching files remotely.
    - **IndexServerImpl.java** – implementation of above interface.
    - **Peer.java** – Main Client logic of starting the peer on the user-provided port number and giving a choice to work serially or parallelly on file sharing.
    - **PeerDownloadInterface.java** – Interface extending Remote, with methods to download at once or in chunks remotely.
    - **PeerImpl.java** – implementation of PeerDownloadInterface methods, describing the main logic of file download serially or in parallel.
- **target** - compiled class files
  - **p2p-pa4-1.0-SNAPSHOT.jar** – executable jar
- **run\_server.sh** – script to start the Indexing Server
- **run\_peer.sh** – script to start Peer
- **generate\_shared\_dir.sh** – script to generate sample data files for usage.

### Run Instructions

- run following script to generate directories and files for peers to register and share.  
`./generate_shared_dir.sh 3 K`  
    //this creates 3 folders inside SHARED\_DIR with random number of files of size in KB  
`./generate_shared_dir.sh 5 M`  
    //this creates 4 folders inside SHARED\_DIR with random number of files of size in KB



- `cd` inside the project location with script files.
- Execute `run_server.sh` to start the Indexing Server  
`./run_server.sh`
- Execute `run_peer.sh` to start a peer  
• `./run_peer.sh`

- Enter port number for the peer
- Enter directory path to register on the Indexing Server
- Choose whether to run download files serially or in parallel
  - 1. Work // assignment 2 – download from a selected peer
  - 2. Work Parallel // assignment 4 – parallel download from multiple peers
- Give an ID to the Peer // say ID=1 for Peer 1 and ID=2 for Peer2.... etc.
- Files are registered with the indexing server:

*Registering file file1\_5 in Indexing Server of size = 262144*  
*Registering file file1\_2 in Indexing Server of size = 89088*  
*Registering file file1\_11 in Indexing Server of size = 212992*  
*Registering file file1\_16 in Indexing Server of size = 114688*

- Make choice to query a file name and download it. //Choose 1 for querying and downloading file
- Enter filename to query
- List of peers hosting the requested file is displayed
- File is downloaded parallelly in chunks from all the peers hosting the file.
- The downloaded file is then checked for integrity by comparing the downloaded file with the original file.
- Exit or repeat

## Implementation and Design Methodology

Design methodology:

1. Indexing Server to register the peers along with their shared directories.
  - We have created a remote interface named "IndexServerInterface" for the index server. This interface extends the Java RMI RemoteException interface. There are two remote methods defined in the interface. The first method is used to register or delete the nodes, and the second method is used to search for a file. Both methods are accessible remotely by peers using Java RMI.
  - We have defined Server classes named "IndexServer" that perform three main tasks: creating an instance of the remote object, binding the instance to a name in the registry and exporting the remote object. The registry is responsible for locating the first remote object that a client needs to use. In our implementation, a static method returns a stub that implements the remote interface java.rmi.registry and sends invocations to the registry on the server's local host on port "3788". Then, the bind method is invoked on the stub to bind the remote object's stub to the name "localhost" in the registry. The "IndexServerImpl" class implements the remote interface, thus providing remote implementations for registry and search.
  - We use a Multi-Values Map data structure where each key has an array of values. These values are File Name, Peer ID, Port Number, and Directory Name. Using these values, the index server, as well as the peers, communicate with each other. Whenever a peer searches for a file, the index server looks up the key and returns a list of all the values. The user then selects from which peer they want to retrieve the file and passes the peerID of that array from the multi-valued map. Using this, the peer retrieves the desired file.
  - First, we need to implement the peers. Each peer must obtain a stub to register on the host, which is the indexing server. To achieve this, we define the methods for file retrieval in "PeerDownloadInterface.java". Then, we implement the method for file downloading in "ClientInterface.java". To do this, we invoke remote methods that are defined in the interface of the index server. When a peer wants to access these remote methods, the user provides the peer ID and directory name for registration on the index server.
2. Accessing/Downloading the requested file parallelly from all the peers hosting the file. This is done as:
  - **Chunking the File**  
 Before initiating the download process, the file to be transferred is divided into fixed-size chunks. In this case, the chunk size is set to 64KB (64 \* 1024 bytes). This ensures uniformity in the size of each chunk.

```
// Download chunks from each peer in parallel
for (int chunkNumber = 0; chunkNumber < numChunks; chunkNumber++) {
    final int chunkIndex = chunkNumber; // Current chunk index
    executor.execute() -> {
        System.out.println("File download in process...");
        try {
            // Get reference to the peer node registry
            ArrayList<String> peerNode = peerNodes.get(chunkIndex % peerNodes.size()); // Select peer node in round-robin fashion
            //0 filename, 1 peerid, 2 port_num, 3 direct
            Registry registry = LocateRegistry.getRegistry( host: "localhost", Integer.parseInt(peerNode.get(2)));
            PeerDownloadInterface pdInter = (PeerDownloadInterface) registry.lookup( name: "root://PeerTest/" + peerNode.get(2) + "/FS");

            // Calculate start offset and end offset for this chunk
            long startOffset = chunkIndex * (long) CHUNK_SIZE;
            long endOffset = Math.min(startOffset + CHUNK_SIZE, fileSize);
            int chunkSize = (int) (endOffset - startOffset);

            // Download chunk from peer
            byte[] chunkData = pdInter.downloadChunk(fileName, startOffset, chunkSize, peerNode.get(3));
            System.out.println("Chunk " + chunkIndex + " downloaded successfully");
            logger.info( msg: chunkData.length + " downloaded from peer " + peerNode.get(1));
            chunkMap.put(chunkIndex, chunkData); // Store downloaded chunk in map
        } catch (Exception e) {
            logger.log(Level.SEVERE, msg: "Error downloading chunk", e);
        }
    }
}
```

- **Parallel Downloading**

The process of downloading a file is made faster by dividing it into several smaller sections, which are then downloaded simultaneously. This is achieved by assigning each section to a peer node in a round-robin fashion. To manage the parallel execution of the download tasks, a thread pool is used. The size of the thread pool is determined based on the number of available peer nodes that can be used for downloading.

- **Chunk Downloading**

To download a file, the system creates a download task for each chunk of the file. The download task specifies the start offset and chunk size to be downloaded. These download tasks are then shared among available peer nodes in a round-robin fashion. This means that each available peer node is assigned chunks in sequential order, rotating among all available peers. The download tasks communicate with the respective peer nodes via RMI to download the assigned chunk of the file.

- **File Reconstruction**

Once all chunks have been successfully downloaded, they are assembled together to reconstruct the original file. The reconstructed file is then registered with the Indexing Server against the respective peer.

```
// Assemble downloaded chunks into a single file
try (FileOutputStream outputStream = new FileOutputStream( name: dirName + File.separator + fileName)) {
    for (int chunkIndex = 0; chunkIndex < numChunks; chunkIndex++) {
        byte[] chunkData = chunkMap.get(chunkIndex);
        if (chunkData != null) {
            outputStream.write(chunkData);
            System.out.println("Chunk " + chunkIndex + " downloaded successfully");
        } else {
            System.out.println("Chunk " + chunkIndex + " not found");
        }
    }
}
```

### 3. Integrity Check

Once a file is downloaded in chunks, the next step is to perform an integrity check to ensure the downloaded data matches the original content of the file. This is usually done by calculating a checksum (here MD5 is used) for both the original chunk and the downloaded chunk, and then comparing them. If the integrity check fails for any chunk, appropriate error handling mechanisms are implemented to handle the discrepancy.

```

private boolean checkIntegrity(String downloadedFilePath, ArrayList<String> peerNode) {
    try {
        // Calculate checksum of the downloaded file
        FileInputStream fis = new FileInputStream(downloadedFilePath);
        MessageDigest md = MessageDigest.getInstance("MD5");
        byte[] buffer = new byte[8192];
        int bytesRead;
        while ((bytesRead = fis.read(buffer)) != -1) {
            md.update(buffer, 0, bytesRead);
        }
        fis.close();
        byte[] downloadedChecksum = md.digest();

        // Calculate checksum of the original file
        Registry registry = LocateRegistry.getRegistry(host: "localhost", Integer.parseInt(peerNode.get(2)));
        PeerDownloadInterface pdInter = (PeerDownloadInterface) registry.lookup(name: "root://PeerTest/" + peerNode.get(2) + "/FS");
        byte[] output = pdInter.FileDownload(peerNode);
        MessageDigest originalMd = MessageDigest.getInstance("MD5");
        originalMd.update(output);
        byte[] originalChecksum = originalMd.digest();

        // Compare checksums
        return MessageDigest.isEqual(downloadedChecksum, originalChecksum);
    }
}

```